

Content-Keyed CRDTs: Convergence, Information Loss, and a Production Pipeline for Entity Deduplication

Author: Subrata Sadhu **Affiliation:** Independent Researcher **Contact:** sadhu.arka507@gmail.com

Abstract

Many distributed systems group concurrent operations by a content-derived key before merging — IPFS by block hash, Git by content hash, entity-resolution systems by name similarity. We formalize this pattern as *content-keyed CRDTs* (CK-CRDTs) and prove four main results: (1) representative-selection via argmax is monotone and content-stable (Theorem 1); (2) canonicalization-at-write-time suffices for no-orphan guarantees under downstream CRDTs (Theorem 2); (3) the information loss is exactly the within-class loser set, and this is a tight lower bound for any merge satisfying our content-key properties (Lemmas 1–2); (4) three properties — determinism, content-locality, and non-key invariance — are necessary and sufficient for convergence under argmax selection, with counterexamples showing each is individually required (Theorem 3). We instantiate the framework as a three-phase projection pipeline for knowledge graphs, evaluate it against naive-merge and ID-at-creation baselines at scales up to 10M operations, and classify Docker, IPFS, Git, Yjs, Automerge, and Loro as instances or non-instances. The convergence model assumes exact-broadcast delivery; delivery-order independence under partial replication is not modeled.

1. Introduction

1.1 The Entity Deduplication Problem

Consider a local-first knowledge graph shared between two agents. Each agent can create named entities (people, projects, concepts) and draw typed edges between them. Neither agent has a global view of the other’s writes. When both agents encounter the same person in different sessions, they each create an entity for that person — with different internal IDs. Naive merge preserves both records, splitting the edge set between them and fragmenting the graph semantically.

This is not a hypothetical edge case. It is the expected steady state in any multi-agent system where agents accumulate knowledge independently. The downstream consequences — split edge sets, fragmented queries, incorrect graph walks — propagate to every operation that references the entity.

1.2 Why Existing Approaches Fall Short

Three classes of solutions exist, each with limitations:

Centralized coordination (mutexes, leader election) requires a global coordinator, contradicting the local-first requirement.

ID-at-creation protocols (Yjs [4], Automerge [5], Loro [6]) assign globally unique identifiers at insertion time. Concurrent creation of the same concept produces two distinct nodes that must be reconciled later. These systems do not collapse duplicates — they accept proliferation and leave deduplication to the application layer.

Post-hoc cleanup (content-addressed systems such as IPFS [7] and Synthing) detects duplicates by content hash after writes propagate, then uses tombstone invalidation and garbage collection. This creates a window of inconsistency where orphan edges exist.

No existing system resolves the problem at projection time by rewriting edge references before they enter the canonical table.

1.3 Contributions

This paper makes four contributions:

1. **Content-Key Monotonicity** (Theorem 1): For argmax representative-selection, monotonicity and content-stability are equivalent — the structural foundation for CK-CRDT merge.
2. **Layered No-Orphan Invariant** (Theorem 2): Canonicalization-at-write-time suffices for no-orphan guarantees when a CK-CRDT is composed with a downstream CRDT that has foreign-key dependencies.
3. **Information-Loss Characterization** (Lemmas 1–2): The information discarded by CK-CRDT merge is exactly the within-class loser set, and this is a tight lower bound — no merge satisfying our content-key properties can discard fewer operations.
4. **Content Key Properties** (Theorem 3): Three properties — determinism, content-locality, and non-key invariance — are necessary and sufficient for convergence under argmax selection, with counterexamples showing each property is individually required.

We additionally instantiate the framework as a three-phase projection pipeline (§5), evaluate it against baselines at scales up to 10M operations (§6), and classify Docker, IPFS, Git, Yjs, Automerger, and Loro as instances or non-instances (§7). The CK-CRDT framework guides the design of any system that groups operations by content before merging: the K1–K3 checklist tells designers exactly what their content key must satisfy, the information-loss lemma quantifies what is discarded, and the classification identifies when content-keying is necessary versus when ID-at-creation suffices.

1.3.1 Companion Paper Relationship

This paper presents the production systems architecture, three-phase projection pipeline, foreign-key edge redirection, and empirical evaluation of CRDT knowledge graph projection. For the formal algebraic framework, content-key monotonicity proofs, composite-key extensions, and complete proofs of Theorems 1–8, we refer the reader to our companion theoretical paper, *A Framework for Content-Keyed CRDT Convergence* [Sadhu, 2026].

1.4 Scope and Assumptions

The convergence model assumes exact-broadcast delivery: all peers eventually receive the same operation set. Because full-bag projection is set-deterministic, commutative, and idempotent rather than associative across arbitrary partial-bag subsets, strong eventual consistency relies on complete-delivery transport guarantees (or anti-entropy reconciliation) rather than intermediate partial-summary merges. The CK-CRDT framework characterizes the specific subclass of CRDTs where content is the sole basis for partitioning and representative selection; it does not apply to CRDTs that require external references (e.g., G-Counters, which read peer IDs and clocks).

2. Background and Definitions

2.1 Standard CRDT Model

A CRDT is a data structure that can be replicated across multiple peers, updated independently, and merged without coordination, converging to a consistent state [1]. Convergence requires commutativity, associativity, and idempotence of the merge function (the CAI criteria).

2.2 Content-Keyed CRDTs

Let $\mathcal{O}$ denote the operation alphabet and K the key space. Each operation $o \in \mathcal{O}$ has content fields $F_C(o)$ and metadata fields $F_M(o)$. The content-key function κ reads only content fields.

Definition 1 (Content Key). A *content key* is a total function $\kappa : \mathcal{O} \rightarrow K$ that depends only on an operation's content fields. The partition $\mathcal{O}/\kappa$ induced by κ defines the equivalence classes under which merge is applied.

Definition 2 (CK-CRDT). A *content-keyed CRDT* is a tuple $(\kappa, \{\rho_k\}, M)$ where $\kappa : \mathcal{O} \rightarrow K$ is a content-key function, $\rho_k : \mathcal{P}(\mathcal{O}_k) \rightarrow \mathcal{O}_k$ is a deterministic representative-selection function for each key k , and M partitions a bag B into per-key classes $C_k(B)$, applies ρ_k to each, and produces $M(B) = \bigcup_{k \in \kappa(B)} \{\rho_k(C_k(B))\}$.

Definition 3 (Winner Set and Redirect Map). The *winner set* is $W(B) = \{\rho_k(C_k(B)) : k \in \kappa(B)\}$. An operation o is a *winner* if $o \in W(B)$; otherwise it is a *loser*. The *redirect map* R maps each loser to its class representative.

3. Main Result 1: Content-Key Monotonicity

Definition 4 (Argmax ρ). A representative-selection function ρ is an *argmax* over a total order $\leq$ on operations if $\rho(S) = \arg \max_{\leq}(S)$ for any non-empty finite S .

Theorem 1 (Content-Key Monotonicity). Let $(\kappa, \{\rho_k\}, M)$ be a CK-CRDT where each ρ_k is an argmax over a total order $\leq$. Then:

- (a) ρ_k is monotone: $S \subseteq S' \implies \rho_k(S') \geq \rho_k(S)$.
- (b) ρ_k is content-stable: $\rho_k(S \cup \{\rho_k(S)\}) = \rho_k(S)$.
- (c) and (b) are equivalent under the argmax premise (neither property alone implies the other for arbitrary ρ).

Proof. ($\implies$) Let $c = \rho_k(S)$. Then $S \cup \{c\} \supseteq S$, so by monotonicity $\rho_k(S \cup \{c\}) \geq c$. Since ρ_k is an argmax over a total order, it selects the unique maximum of $S \cup \{c\}$. Since c is already the maximum of S (by definition $c = \rho_k(S)$), no element of S exceeds c . Therefore $\rho_k(S \cup \{c\}) = c$. ($\impliedby$) Let $S \subseteq S'$ and $c' = \rho_k(S')$. Since $S \subseteq S'$, $\rho_k(S) \in S'$. If $\rho_k(S) > c'$, then $\rho_k(S) \in S'$ and $\rho_k(S) > \rho_k(S')$, contradicting the argmax property (which requires $\rho_k(S')$ to be the maximum of S'). Therefore $\rho_k(S') \geq \rho_k(S)$. $\square$

Corollary 1. In our pipeline, $\rho_k(S) = \max(S)$, which is an argmax over the natural total order on IDs.

4. Main Result 2: Layered No-Orphan Invariant

Definition 5 (Foreign-Key Dependency). A downstream CRDT M_{down} has a *foreign-key dependency* on an upstream CK-CRDT M_{CK} if M_{down} 's operations include fields whose values are entity IDs produced by M_{CK} .

Theorem 2 (Layered No-Orphan Invariant). Let M_{CK} be a fully merged CK-CRDT producing canonical IDs via $W(B)$. Let M_{down} be a downstream CRDT with foreign-key dependencies. If M_{down} applies the canonical redirect function R_{id} to all endpoints at write time, then every edge endpoint references an entity in $W(B)$.

Proof. For any endpoint e : if e was a loser, $R_{\text{id}}(e)$ maps to a canonical ID in $W(B)$. If e was already canonical, $R_{\text{id}}(e) = e \in W(B)$. $\square$

5. Main Result 3: Information Loss

Lemma 1 (Kernel of CK-Merge). The merge M_{CK} is many-to-one over each equivalence class. Two operation sets produce the same output iff for every key-class C_k , the representative $\rho_k(C_k \cap O_1) = \rho_k(C_k \cap O_2)$. The information discarded is exactly the within-class loser set $O \setminus W(O)$.

Lemma 2 (Information-Loss Lower Bound). For any CK-CRDT $(\kappa, \{\rho_k\}, M)$ satisfying (K1)–(K3), the merge discards at least $|O| - |\kappa(O)|$ operations, and this bound is tight.

Proof. The canonical state contains at most one representative per key class. Since there are $|\kappa(O)|$ distinct keys, $|\Sigma| \leq |\kappa(O)|$, so at least $|O| - |\kappa(O)|$ operations are discarded. CK-CRDT merge achieves exactly this: ρ_k maps each non-empty class to one element (by Definition 2), producing exactly $|\kappa(O)|$ representatives. The tightness depends on two independent facts: (i) the structural constraint that $\rho_k : \mathcal{P}(\mathcal{O}_k) \rightarrow \mathcal{O}_k$ selects one representative per class (Definition 2), and (ii) K1–K3 ensure the partition into classes is deterministic and content-local, so no class is split or duplicated across peers. $\square$

6. Main Result 4: Content Key Properties

We define three properties of the content key κ :

(K1) Determinism: $\kappa(o)$ is the same on every peer for the same operation.

(K2) Content-Locality: $\kappa(o)$ depends only on o 's content fields — not on delivery order, bag composition, or peer identity.

(K3) Non-Key Invariance: Updating a non-key field does not change $\kappa(o)$.

Theorem 3 (Convergence — Necessity and Sufficiency). The properties (K1)–(K3) are necessary and sufficient for convergence under argmax selection. Specifically:

(Sufficiency.) If κ satisfies (K1)–(K3) and each ρ_k is an argmax, then M converges: all peers with the same operation bag produce the same canonical state.

(*Necessity.*) Violating any one of (K1)–(K3) while satisfying the other two permits either divergence (different peers produce different outputs) or correctness degradation (convergence holds but entity deduplication fails).

Proof (sufficiency). (K1) ensures all peers compute the same partition $\kappa(B)$. (K2) ensures the partition is invariant under delivery order. (K3) ensures metadata updates don’t shift keys. Given a stable partition, the binary merge $m_k(o_1, o_2) = \rho_k(\{o_1, o_2\})$ satisfies CAI under argmax: commutative by set symmetry ($\{o_1, o_2\} = \{o_2, o_1\}$), associative by Theorem 1 ($m_k(m_k(o_1, o_2), o_3) = \rho_k(\{\rho_k(\{o_1, o_2\}), o_3\}) = \rho_k(\{o_1, o_2, o_3\})$ since argmax of a set is order-independent), and idempotent by Theorem 1(b). The union of independent per-class CAI merges is CAI: classes are disjoint and processed independently, so commutativity and associativity hold across classes. $\square$

Proof (necessity — sketch). - *K1 violation:* Non-deterministic key $\rightarrow$ different peers produce different partitions $\rightarrow$ divergence. Example: $B = \{o, o'\}$ with $\kappa_A(o) = \kappa_A(o') = k_1$ but $\kappa_B(o) = k_1, \kappa_B(o') = k_2$. Then $|M_A(B)| = 1 \neq 2 = |M_B(B)|$. Verified: **TestK1Violation**. - *K2 violation:* Key depends on bag size $\rightarrow$ different delivery orders yield different keys $\rightarrow$ different partitions $\rightarrow$ divergence. Example: $\kappa(o) = k_a$ if $|B| = 1$ and $\kappa(o) = k_b$ if $|B| > 1$. Verified: **TestK2Violation**. - *K3 violation:* Key reads non-key field $\rightarrow$ semantic duplicate. Example: o has (name=“alice”, type=“person”) with $\kappa(o) = k_1$; o' adds description=“lawyer” (a non-key update), but $\kappa(o') = k_2$ because the key derivation reads description. Then $M(B) = \{o, o'\}$ — two entities instead of one. Convergence holds (same bag $\rightarrow$ same output), but entity deduplication fails. Verified: **TestK3Violation**. $\square$

7. Three-Phase Projection Pipeline

We instantiate the CK-CRDT framework as a three-phase pipeline for knowledge graphs.

Phase 1 — Entity Merge. Entity operations are merged using a 2P-Set for membership (tombstoned if any remove dominates any add) and LWW-Register per metadata field (name, type, description). The output is a merged entity state σ_E .

Phase 2 — Canonical Entity Resolution. Entities in σ_E are grouped by inception fingerprint — a SHA-256 hash of (name, type, description) computed at creation time. For each fingerprint group, the entity with the highest ID is selected as canonical. A redirect map R records which IDs were merged into which winners.

Phase 3 — Edge Projection with Redirect. Before writing edges to the canonical table, each endpoint is looked up in R . Loser IDs are rewritten to winner IDs. An orphan guard drops any edge referencing a non-canonical entity.

Algorithm 1: Three-Phase Projection

Input: Operation logs O_E (entity ops), O_{Ev} (edge ops)

Output: Canonical entities Σ , canonical edges σ'_{Ev} , redirect map R

Phase 1: $\sigma_E \leftarrow \text{merge_entity_ops}(O_E)$

 for each entity_id in σ_E :

 apply 2P-Set: tombstone if any remove dominates any add

 apply LWW: select winner per field (name, type, description)

```

Phase 2: ( $\Sigma$ , R)  $\leftarrow$  entity_dedup(sigma_E)
  for each fingerprint group F:
    winner  $\leftarrow$  max(F) // by entity_id
    for each loser in F \ {winner}: R[loser]  $\leftarrow$  winner

Phase 3: sigma'_Ev  $\leftarrow$  merge_edge_ops(O_Ev)
  for each edge endpoint e in sigma'_Ev:
    if e in domain(R): e  $\leftarrow$  R[e] // rewrite loser to winner
  sigma'_Ev  $\leftarrow$  orphan_guard(sigma'_Ev) // drop non-canonical endpoints

return  $\Sigma$ , sigma'_Ev, R

```

Convergence. Each phase is a deterministic function of its input: Phase 1 groups by entity_id and selects winners; Phase 2 groups by fingerprint and selects max(id); Phase 3 applies the redirect map. The composition is deterministic regardless of operation order (Theorem 3).

No-orphan invariant. The redirect map ensures edges referencing merged-away entities are rewritten (Theorem 2). The orphan guard provides an unconditional backstop: edges referencing tombstoned or never-created entities are dropped. Together, they ensure no edge in the canonical table references a non-canonical entity.

Convergence model. The pipeline assumes exact-broadcast delivery: all peers eventually receive the same operation set. The proof shows delivery-order independence under this model. Partial replication is not modeled.

8. Evaluation

8.1 Baseline Comparison

We compare three approaches on 5,000 concurrent entity ops with 50 distinct entities:

Metric	Naive (ID-at-creation)	Redirect-only	Full pipeline
Canonical entities	5,000	50	50
Semantic duplicates	4,950	0	0
Orphan edges	460	460	0
Redirect map entries	0	4,950	4,950
Overhead vs naive	—	+23%	+35%

The naive merge (equivalent to Yjs/Automerger semantics) preserves all duplicates and orphan edges. The full pipeline eliminates both at ~35% overhead — dominated by Phase 1 entity merge (~94% of runtime), not by content-keyed dedup.

8.2 Scaling

Wall-clock time grows linearly with N (merge + dedup, no SQLite I/O). At $K=1000$ (realistic workload: 1000 distinct entities):

N	Throughput	Time
100K	271K ops/s	0.37s
1M	247K ops/s	4.0s
10M	138K ops/s	72s

Throughput degrades 1.96x from 100K to 10M, attributable to Python dict overhead — not algorithmic. The full pipeline with SQLite I/O shows comparable throughput (192K→274K ops/s), confirming SQLite is not the bottleneck.

8.3 Adversarial Robustness

The pipeline was tested against 35 test scenarios across 10 categories, including 14 genuinely adversarial tests (Byzantine version vectors, fingerprint collision attacks, malicious peer behavior, clock skew) and 21 standard robustness tests (boundary cases, concurrency, edge conditions). Key results:

- **Fingerprint collision:** 10,000 ops on a single fingerprint group merge in <0.1s — graceful degradation, no crash.
- **K1-necessity counterexample:** Two peers using different normalizations produce different fingerprints → divergence confirmed.
- **VV overflow:** Counters at 10^9 — dominance check still correct.

8.4 Production Path with SQLite

The full pipeline (`project_crdt_to_entities`) includes SQLite reads/writes. At K=10, throughput is 192K→274K ops/s from 1K to 1M — comparable to in-memory merge+dedup. SQLite I/O is not the bottleneck; entity merge (Phase 1) dominates at ~94% of runtime.

9. Classification of Real Systems

System	Category	Key κ	K1	K2	K3	Notes
Our pipeline	CK-CRDT	SHA-256(name, type, desc)	Y	Y	Y	Canonical example; max-ID representative
Docker/OCI layers	CK-CRDT	SHA-256(layer content)	Y	Y	Y*	*K3 vacuous (layers immutable); no CRDT merge

System	Category	Key κ	K1	K2	K3	Notes
IPFS/IPLD	Content-addressed	SHA-256(content)	Y	Y	Y*	No merge function; related but not CK-CRDT
Git (blobs)	Content-addressed	SHA-1(content)	Y	Y	Y*	Blob dedup is CK-CRDT-like; commit merge is 3-way
Syncthing	Content-addressed	Block hash	Y	Y	Y*	Block-sync; file-level merge via timestamps
Yjs	ID-at-creation	Client-generated clock ID	—	—	—	Position-tracking requires ID-at-creation
Automerge	ID-at-creation	UUID at creation	—	—	—	Same pattern; sequence CRDT
Loro	ID-at-creation	Random ID at creation	—	—	—	Delta-CRDT with ID-at-creation

Docker as CK-CRDT instance. Docker’s storage driver deduplicates layers by content hash. This is a CK-CRDT: $\kappa = \text{SHA-256}(\text{layer content})$, K1–K3 hold (K3 vacuous: layers are immutable). Docker did not design this as a CK-CRDT; the framework classifies it post-hoc. Docker does not provide a redirect map or convergence guarantee across independent registries — a CK-CRDT formulation would add these properties.

10. Discussion

10.1 When Content-Keying Is Required

Content-keying is necessary when multiple peers create semantically identical entities independently and the system must collapse duplicates at merge time. ID-at-creation suffices when entities are unique by construction and position-tracking requires stable identities (collaborative text editing). The framework identifies exactly which systems fall into each category.

10.2 Where the Framework Breaks

Three failure modes:

1. **Content-key collisions.** “Java” (programming language) and “Java” (island) share all content fields $\rightarrow$ incorrectly merged. Mitigated by composite keys (Theorem 4 in §11).
2. **Cross-class causal dependencies.** Entity creation and edge referencing have causal dependencies not modeled by the independence assumption. Theorem 2 addresses foreign-key redirects; general cross-class causality is open.
3. **Adaptive key cycles.** If key migration creates a cycle, convergence may break (Theorem 6 in §11).

10.3 False Merges

The false merge rate depends on the domain. In agent memory systems, descriptions typically contain distinguishing context (“programming language for Android” vs “largest island in Southeast Asia”), making false merges rare. In sparse-description domains, the rate increases. The framework addresses this via composite keys (Theorem 4): extending the key with additional fields reduces false merges. Empirical measurement on real knowledge graph dumps is future work.

10.4 Orphan Guard Tradeoff

The orphan guard achieves zero orphans by silently dropping edges to non-canonical entities — a deliberate tradeoff favoring invariant enforcement over edge preservation. The convergence model (Theorem 3) applies to the operation set; the orphan guard is a post-merge filter outside the formal convergence model. In production, an edge to a tombstoned or never-created entity is semantically meaningless, so silent dropping is the correct behavior.

11. Extensions

The following results extend the framework. Proofs are in the companion paper [9].

Composite keys (Theorem 4). If $\kappa' = (\kappa_1, \kappa_2)$ where each κ_i satisfies K1–K3, then κ' satisfies K1–K3. Our pipeline’s fingerprint $\kappa(o) = \text{SHA-256}(\text{name}, \text{type}, \text{description})$ is a composite key with three components.

Approximate keys (Theorem 5). Deterministic approximate keys (e.g., Levenshtein-based) converge if they satisfy K1. Non-deterministic ones fail by K1 violation.

Adaptive keys (Theorem 6). Keys that evolve over time converge if the migration graph is acyclic and deterministic. Cycles may break convergence.

Delta-CRDT composition (Theorem 7). CK-CRDTs compose with delta-CRDTs when the delta computation depends only on the merge output, not on the raw operation bag.

12. Related Work

CRDT foundations. Shapiro et al. [1] define CAI convergence and classify CRDTs. CK-CRDTs are a restricted join: the merge computes the join within each content-key class, then takes the union across classes. This preserves join-semilattice properties because each class is processed independently.

Record linkage. Fellegi–Sunter [2] and Cohen et al. [3] address record linkage using probabilistic matching. Our CK-CRDT framework extends the keying idea to the distributed, concurrent setting where multiple peers create records independently.

Content-addressed storage. IPFS [7] and Git [8] use content hashes but lack CRDT merge functions. Synthing uses block-level content addressing with file-level merge. These systems illustrate the keying pattern but fall outside the CK-CRDT framework.

Collaborative editing. Yjs [4], Automerge [5], and Loro [6] use ID-at-creation for position stability. Content-keying would collapse operations at different positions with identical content, breaking sequence semantics.

13. Conclusion

We formalized content-keyed CRDTs and proved four main results: argmax monotonicity (Theorem 1), layered no-orphan composition (Theorem 2), tight information-loss bounds (Lemmas 1–2), and necessary-and-sufficient convergence conditions (Theorem 3). The framework classifies Docker, IPFS, Git, Yjs, Automerge, and Loro, explaining when content-keying is necessary and when ID-at-creation suffices. We instantiated the framework as a three-phase projection pipeline, evaluated it at 10M operations against naive-merge and ID-at-creation baselines, and verified robustness with 35 test scenarios (14 adversarial, 21 standard). The K1–K3 checklist provides a concrete design tool for any system that groups operations by content before merging.

References

- [1] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-Free Replicated Data Types,” in *Stabilization, Safety, and Security of Distributed Systems*, LNCS 6976, Springer, 2011, pp. 386–400.
- [2] I. P. Fellegi and A. B. Sunter, “A Theory for Record Linkage,” *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, 1969.
- [3] W. W. Cohen, P. Ravikumar, and S. E. Fienberg, “A Comparison of String Distance Metrics for Name-Matching Tasks,” in *Proceedings of IJCAI 2003*, 2003, pp. 73–77.
- [4] P. Nicolaescu, K. Jahns, M. Derntl, and R. Klamma, “Yjs: A Framework for Near Real-Time P2P Shared Editing on Arbitrary Data Types,” in *Proceedings of ICWE 2015*, LNCS 9114, Springer, 2015, pp. 675–678.
- [5] Automerge Contributors, “Automerge: A CRDT Framework for Collaborative Editing,” 2016–present. <https://github.com/automerge/automerge>

- [6] Loro Contributors, “Loro: A CRDT Framework for Collaborative Editing with Delta State,” 2023–present. <https://github.com/loro-dev/loro>
- [7] J. Benet, “IPFS - Content Addressed, Versioned, P2P File System,” arXiv:1407.3561, 2014.
- [8] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014.
- [9] S. Sadhu, “A Framework for Content-Keyed CRDT Convergence,” preprint, 2026.